

SEN381 ASSIGNMENT 1

RESEARCH FOUNDATIONS FOR SOFTWARE
ENGINEERING DECISIONS

GROUP E

Contents

Section 1: Question 1 - Software Engineering Beyond Programming [10 Marks]	2
Section 2: Question 2 - Thinking Ahead: Decisions Across the Software Lifecycle [15 Marks]	3
Section 3: Question 3 - Evidence-Based Technology Stack Decisions [15 Marks].....	11
Section 4: Question 4 - Responsible AI in Software Engineering [10 Marks].....	16
Section 5: Academic Reference List.....	17
Section 6: Appendix - AI Research & Verification Record.....	18
Section 7: Submission & Integrity Checklist	Error! Bookmark not defined.

Section 1: Question 1 - Software Engineering Beyond Programming [10 Marks]

1.1 Beyond "Just Coding": The Multi-Dimensional Engineering Challenge

When you are creating a system there are several things that you must consider especially if you are creating it for a consumer and/ or being contracted to make it. any change or adjustment to what the customer needs requires you to update many things, like documentation it could change the schedule of the project, it could even affect the quality of the system. You must consider nearly everything before you can start creating the system especially if it is a life critical system since if one thing fails it could cost and endanger the lives of many people for instance the Boeing 737 max crashes. (Johnston, P, 2019)

1.2 Systemic Interaction of the Six Core Engineering Concerns

There are 6 things to consider before, during and after a software engineering undertaking:

- Stakeholder needs and requirements
- Scope
- Schedule
- Cost and resources
- Quality
- Engineering risk

All these considerations are deeply intertwined, even a perceived small change to the scope can drastically alter the course of the project so it is vitally important to create systemised and understood process for change and create comprehensive plans for the whole lifecycle of the project.

1.3 Empirical Case Study Analysis: Boeing 737 MAX MCAS Failure

The cause of the Boeing 737 max crashes wasn't a MCAS software failure; the software was working properly according to the data given however the primary cause of the incidents were firstly the rushed production (Schedule constraint) and design process causing the quality and completeness of the documentation and training to be severely lacking. Secondly because Boeing wished for the new design to be a direct upgrade to the Boeing 737 curtailing a lot of the proper certification, safety checks and testing that should be in place. (Johnston, P, 2019)

The decision to speed up design and forgo proper testing and safety practices allowed the project to successfully be released earlier however this decision made the plane unsafe for pilots to fly causing, after about 2 years of commercial flight, the deaths of a total of 346 passengers. (Johnston, P, 2019)

Section 2: Question 2 - Thinking Ahead: Decisions Across the Software Lifecycle [15 Marks]

2.1 The Engineering Horizon: Early Consideration versus Late Implementation

Decisions need to be made well before the code that implements them is written. Software engineers are required to evaluate, model and bound big technical decisions several phases prior to the lifecycle sprint where code is written or where runtime operational consequences manifest (Krutchten, 2001; Bass, Clements and Kazman, 2021).

From an economic & lifecycle perspective, this practice in Boehm's Cost of Change Curve (Boehm, 1981; Boehm and Basili, 2001). When an architectural issue such as transaction boundaries, trust boundaries, or deployment parity is examined during the inception and requirements baseline stages, the cost of exploring alternatives and refining models remains minimal. However, if that same decision is overlooked and postponed until construction or release, the cost rises because existing codebases, database schemas, automated tests, and interface contracts have already become fixed around unexamined assumptions (Boehm and Basili; IEEE Computer Society, 2014).

As noted earlier in international engineering standards, "Software engineering differs from ad-hoc programming because it proactively manages the temporal gap between decision inception and operational consequence, preserving future engineering options while minimising rework debt" (IEEE Computer Society, 2014). Proactive forward engineering requires establishing structured decision records early to prevent catastrophic architectural lock-in and debt.

2.2 Deep-Dive Analysis of Six Significant Early Engineering Decisions

2.2.1 Decision 1: Architectural Modularity & System Boundaries

- **What the decision/concern is:**
Choosing the macro-architectural decomposition style (Monolithic, Modular Monolith, or Microservices) and defining formal module boundaries. This involves splitting the problem domain into cohesive, loosely coupled bounded contexts and defining module interfaces based on the principle of information hiding (Parnas, 1972; Bass, Clements and Kazman, 2021).
- **When it should first be considered:**
During Milestone 1 (Inception & Requirements Baseline), following domain modelling and stakeholder workflow analysis.
- **When it is likely to be implemented or become operationally significant:**
Built during Milestones 2 & 3 (Component Construction) and becomes operationally critical during Milestone 4 (Production Scaling, Continuous Integration, and Independent Service Deployment).
- **What information is needed to make a sound decision:**
Domain complexity, anticipated peak transaction volume, latency budgets, team size and organisational communication structure (aligning with Conway's Law), and business requirements for independent deployment cadences (Bass, Clements and Kazman, 2021).

- **Which later activities it enables, constrains, or affects:**
Explicitly define the database persistence topology (shared single database vs. isolated schemas), repository structure (monorepo vs. polyrepo), API contract specifications, CI/CD pipeline complexity, and automated integration test boundaries.
- **What could happen if consideration is postponed until implementation time:**
Without upfront modular boundaries, the codebase degrades into a tightly coupled ‘Big Ball of Mud.’ Resolving tangled dependencies post-construction requires multi-month code rewrites, unravelling circular class references, and executing high-risk database schema splits that delay project delivery and cause widespread regression defects (Krutchten, 2001; Fowler, 2018).
- **Which credible evidence supports the analysis:**
Parnas (1972, p. 1056) establishes: *“We propose instead that one begins with a list of difficult design decisions or design decisions which are likely to change. Each module is then designed to hide such a decision from the others.”* Supported further by Bass, Clements and Kazman (2021, p. 15) who note: *“Architecture is the set of design decisions which, if made incorrectly, are the hardest and most expensive to change later in the lifecycle.”*

2.2.2 Decision 2: Data Persistence Strategy & Schema Evolution

- **What the decision/concern is:**
Choosing the fundamental data storage paradigm (Relational/SQL enforcing strict ACID transactional guarantees vs. Document/NoSQL providing BASE eventual consistency) and designing the automated, version-controlled schema migration and audit logging strategy (Kleppmann, 2017).
- **When it should first be considered:**
In Milestone 1 (Requirements Baseline & Conceptual Design), as soon as entity relationships, data volume, and transactional integrity rules become known.
- **When it is likely to be implemented or become operationally significant:**
Built during Milestones 2 & 3 (Data Access Layer & API Development), and becomes operationally critical during concurrent multi-user load, live schema migrations, and disaster recovery operations in Milestone 4.
- **What information is needed to make a sound decision:**
Entity relationship cardinality, read-to-write ratios, statutory ACID compliance mandates (e.g., immutable audit trails for state transitions), concurrent write lock contention tolerance, and regulatory data retention requirements (Kleppmann, 2017).
- **Which later activities it enables, constrains, or affects:**
Shapes Object-Relational Mapping (ORM) setup, database indexing strategies, API data transfer objects (DTOs), transactional locking behaviour, and automated test database seeding routines.
- **What could happen if consideration is postponed until implementation time:**
Leads to unindexed full-table scans, database deadlocks, and data corruption during concurrent write operations. Moving live relational data to a different storage model after launch requires high-risk ETL scripts, schema lockouts, and prolonged system downtime (Kleppmann, 2017).

- **Which credible evidence supports the analysis:**
Kleppmann (2017, p. 4) notes: “Data outlives code; shifting a persistence model and aligning live relational state post-deployment ranks among the riskiest and most costly operations in software engineering.”

2.2.3 Decision 3: Authentication, Authorization & Trust Boundaries (Security by Design)

- **What the decision/concern is:**
Designing the security perimeter, authentication methods (JWT, OAuth2, OpenID Connect), Role-Based Access Control (RBAC) privilege models, trust boundaries, and cryptographic secrets management following Security by Design principles (OWASP Foundation, 2021; NIST, 2022).
- **When it should first be considered:**
In Milestone 1 (Scope Baseline & Threat Modeling), alongside stakeholder persona elicitation and sensitive data classification.
- **When it is likely to be implemented or become operationally significant:**
When it is likely to be implemented or become operationally relevant:
Implemented across Milestones 2 & 3 (API Middleware, Controller Guards, and UI Route Protection), and operationally validated during security regression testing, vulnerability scanning, and production deployment in Milestone 4.
- **What information is needed to make a sound decision:**
User role hierarchy matrices, data classification levels (public vs. sensitive PII), API exposure perimeter, external Identity Providers, and statutory privacy regulations such as POPIA and GDPR (NIST, 2022).
- **Which later activities it enables, constrains, or affects:**
Governs API route architecture, controller middleware interceptors, database security schema design (tenant/role tables), client token storage, and automated security penetration test suites.
- **What could happen if consideration is postponed until implementation time:**
Attempting to ‘bolt on’ security at release invariably introduces critical vulnerabilities such as Broken Object Level Authorization (BOLA / OWASP API #1). Fixing this late requires manually retrofitting authorization checks across every controller, significantly increasing human error and regulatory non-compliance risk (OWASP Foundation, 2021; NIST, 2022).
- **Which credible evidence supports the analysis:**
NIST (2022, p. 5) mandates: ‘Security cannot be inspected or tested into a system after construction; it must be an intentional architectural constraint established before the first line of code is committed.’ Further supported by the OWASP Foundation (2021) Top 10 Security Risks.

2.2.4 Decision 4: Automated Testing Strategy & Testability Architecture

- **What the decision/concern is:**
Defining the automated verification framework, which includes the quantitative distribution of the Testing Pyramid (fast Unit tests, isolated Integration tests, and End-to-End tests), outlining Dependency Inversion boundaries

to support test doubles and mocking, and establishing mandatory Continuous Integration (CI) quality gates (Fowler, 2018; Humble and Farley, 2010).

- **When it should first be considered:**
During Milestone 2 (Architecture & Component Interface Design), prior to substantive application coding.
- **When it is likely to be implemented or become operationally significant:**
Implemented continuously throughout Milestone 3 (Construction Sprints and Pull Request Verification) and serves as the primary automated quality gate governing release readiness in Milestone 4.
- **What information is needed to make a sound decision:**
The importance of core domain logic, external third-party API dependencies that need test doubles or mocks, CI build time limits (under 5 minutes), and acceptable defect tolerance levels (Humble and Farley, 2010; IEEE Computer Society, 2014).
- **Which later activities it enables, constrains, or affects:**
It directly shapes component design patterns (requiring Dependency Injection and interface abstractions), Pull Request merge standards, developer refactoring speed, and automated deployment pipeline triggers.
- **What could happen if consideration is postponed until implementation time:**
If developers write classes that directly instantiate heavy concrete dependencies, the code cannot be tested in isolation. Teams become stuck with slow, fragile manual testing, allowing serious regression defects to reach production and slowing deployment speed (Fowler, 2018; Humble and Farley, 2010).
- **Which credible evidence supports the analysis:**
Fowler (2018, p. 45) states: “Testability is an architectural property; code written without testability in mind is almost impossible to test effectively without undergoing massive structural refactoring.” This is reinforced by Humble and Farley (2010) in Continuous Delivery.

2.2.5 Decision 5: Deployment Target, Environment Parity & Configuration Management

- **What the decision/concern is:**
Choosing the production hosting environment (Linux Containers/Docker, Cloud PaaS, Serverless) and enforcing 12-Factor App methodology—strictly separating environment configuration from executable code and ensuring consistency across Development, Testing, Staging, and Production environments (Wiggins, 2017; Humble and Farley, 2010).
- **When it should first be considered:**
During Milestones 1 & 2 (Constraints Analysis & Architectural Baseline).
- **When it is likely to be implemented or become operationally significant:**
Configured during Milestone 3 (Staging Deployment and CI/CD Automation) and fully tested during Milestone 4 (Production Deployment, Scaling, and Disaster Recovery).
- **What information is needed to make a sound decision:**
Budgets for infrastructure hosting costs, compute resource limits (CPU/RAM thresholds

on target tiers), platform OS runtime compatibility (e.g., Linux container compatibility), and requirements for integrating backing services (Wiggins, 2017).

- **Which later activities it enables, constrains, or affects:**
Governs container image build packaging (Docker files), configuration and secrets injection methods, file I/O operations (requiring stateless file handling through object storage instead of local disk), and continuous deployment automation.
- **What could happen if consideration is postponed until implementation time:**
Not preparing for the deployment target early brings about the widespread "works on my machine" failure mode. Developers write code that depends on local operating system dependencies, hardcoded file paths, or Windows-specific libraries that fail right away when deployed to a Linux-based production container (Humble and Farley, 2010; Wiggins, 2017).
- **Which credible evidence supports the analysis:**
Humble and Farley (2010, p. 258) state: *"Environment parity ensures that software developed locally behaves identically in production, eliminating the friction and unreliability of environment-specific deployment bugs."* Supported by Wiggins (2017) on *The Twelve-Factor App*.

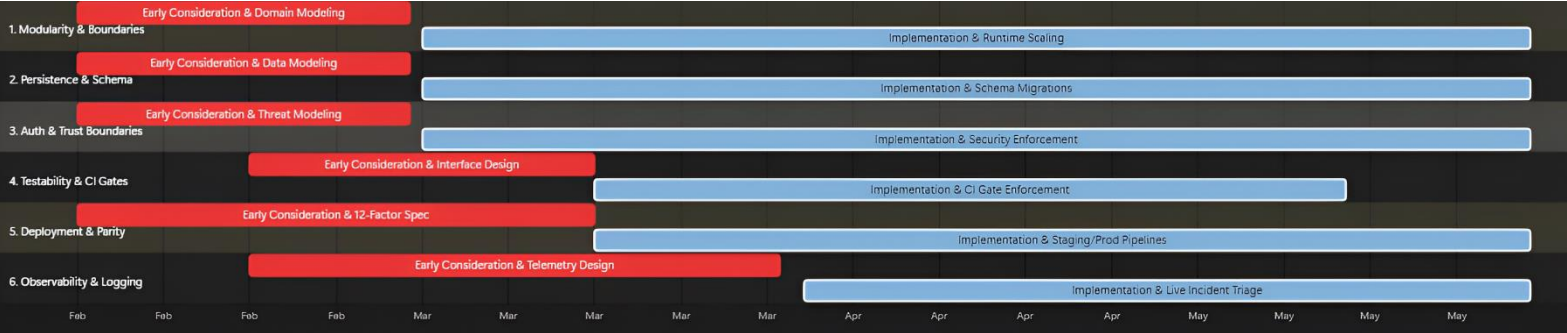
2.2.6 Decision 6: Observability, Structured Telemetry & Diagnostics

- **What the decision/concern is:**
Designing the operational telemetry architecture, including structured JSON event logging, distributed request correlation IDs (tracing requests across frontend and backend boundaries), health-check probes (/health), metrics collection, and privacy-compliant error logging (Beyer et al., 2016).
- **When it should first be considered:**
During Milestone 2 (API Architecture & Cross-Cutting Architectural Design).
- **When it is likely to be implemented or become operationally significant:**
Implemented in middleware during Milestone 3 (Construction & Staging Validation) and becomes operationally vital during Milestone 4 (Production Monitoring, Live Incident Response, and Service Level Agreement Tracking).
- **What information is needed to make a sound decision:**
Defined Service Level Indicators (SLIs), Service Level Objectives (SLOs), log storage cost limits, diagnostic verbosity requirements, and compliance rules for scrubbing PII and passwords from log streams (Beyer et al., 2016; NIST, 2022).
- **Which later activities it enables, constrains, or affects:**
Directly shape global HTTP interceptors, exception handling wrappers, database query execution logging, production alerting mechanisms, and operational incident triage workflows.
- **What could happen if consideration is postponed until implementation time:**
Postponing telemetry results in total "Operational Blindness." When intermittent production outages, slow queries, or unauthorized access attempts occur, support teams cannot reconstruct the sequence of events. Developers are forced to manually inject ad-hoc logging into live production systems, leading to severely inflated Mean Time to Recovery (MTTR) (Beyer et al., 2016).

- Which credible evidence supports the analysis:
Beyer et al. (2016, p. 58) in Google’s *Site Reliability Engineering* state: "Observability is not something you add to a completed system; without built-in telemetry and correlation tracing, diagnosing complex production failures is reduced to guesswork."

2.3 Visual Artefact: The Decision Horizon Map

Early Architectural Consideration vs. Downstream Implementation and Operational Impact across the Software Engineering Lifecycle.



The Decision Horizon Map visualises the essential separation between the Early Consideration Horizon, and the Implementation / Operational Horizon. The engineering team avoids technical risk by evaluating trade-offs, abstractions and boundaries during the foundation phase. Construction in Milestone 3 proceeds against verified contracts preventing the emergency rework and schedule failure that occur when decisions are deferred.

Lifecycle Decision Horizon Cross-Reference Table

Decision / Concern	Early consideration Horizon (analyse & model)	Implementation Horizon (Code Construction)	Operational Horizon (Runtime & Scaling)	Downstream Dependencies Enabled
1. Modularity & Boundaries	Milestone 1 (Requirements Baseline & Domain Elicitation)	Milestone 2 & 3 (Module Interfaces & Component Build)	Milestone 4 & Operations (System Scaling & Feature Evolution)	Dictates schema boundaries, teamwork breakdown, API contracts, and CI pipeline setup.
2. Persistence & Schema	Milestone 1 (Conceptual Data Modelling & ACID/BASE Trade-offs)	Milestone 2 & 3 (Entity Mapping, Migrations & DAL Code)	Milestone 4 & Operations (Concurrent Transactions, Live Migrations)	Enables data integrity, query optimization, automated test database seeding, and backup routines.
3. Auth & Trust Boundaries	Milestone 1 (Scope Baseline, Role Modelling & Threat Modelling)	Milestone 2 & 3 (Auth Middleware, Tokens & Route Guards)	Milestone 4 & Operations (Security Audits, Pen Testing, Compliance)	Establishes zero-trust API protection, data privacy compliance (POPIA), and RBAC access tables.
4. Testability & CI Gates	Milestone 2 (Component Interface Design & DI Boundaries)	Milestone 3 (Unit/Integration Test Authoring & CI Gates)	Milestone 4 & Operations (Regression Safety & Release Sign-Off)	Enables rapid developer refactoring, automated PR validation, and high release velocity.
5. Deployment & Parity	Milestone 1 & 2 (Hosting Constraints & 12-Factor Spec)	Milestone 3 (Containerization, IaC & Staging Pipelines)	Milestone 4 & Operations (Zero-Downtime Deployment & Rollback)	Eliminates environment drift, guarantees Linux container compatibility, and controls cloud costs.
6. Observability & Logging	Milestone 2 (API Logging Contracts & Correlation IDs)	Milestone 3 (Middleware Filters & Structured JSON Logs)	Milestone 4 & Operations (Incident Triage, Alerting & SLA Tracking)	Dramatically reduces MTTR, enables proactive error detection, and provides auditable activity logs.

2.4 Critical Synthesis: Why waiting for Implementation is already too late.

Delaying foundational decisions until an activity is due for implementation will consistently cause project failure. In software engineering, “implementation is not the beginning of decision-making; it is the physical realisation of decisions that have already been evaluated, traded off, and bounded (Krutchten, 2001; Bass, Clements and Kazmand, 2021). When an engineering team postpones foundational concerns until the construction sprint in which that feature is coded, four severe structural failure mechanisms are set off:

1. Pre-Existing Architectural Lock-in and Structural Inertia:

During early construction, developers will make implicit assumptions to write immediate functional code. If authentication / data consistency has not been specified, developers default to the easiest local solution (e.g., hardcoding database queries directly into UI controllers). By the time the team formally attempts to implement security or testability during late construction, hundreds of lines of dependent code have already solidified around those unverified assumptions. Dismantling this established code creates immense organizational resistance, developer fatigue, and catastrophic project friction (Bass, Clements and Kazman, 2021; Fowler, 2018).

2. Combinatorial Rework and Interface Divergence:

Software modules communicate with one another through shared interfaces. Following the modularity principles laid out by Parnas (1972), stable interfaces are what make parallel development possible. If fundamental choices about persistence transactions or component decoupling are put off, different developers end up making conflicting technical assumptions. When these modules come together during system integration, the interfaces do not line up properly. Fixing this mismatch means tearing down working code in several subsystems at once, which triggers a combinatorial explosion of rework and invalidates earlier testing (Parnas, 1972; Boehm and Basili, 2001).

3. Deadline Pressure and the Technical Debt Trap:

When unresolved architectural gaps surface during implementation or testing sprints, the team is inevitably working under tight schedule constraints. With severe deadline pressure, engineers lack the time to carry out clean architectural redesigns. Instead, they are compelled to apply quick, fragile fixes—such as bypassing automated quality gates, turning off SSL verification, or writing monolithic database queries (Humble and Farley, 2010; NIST, 2022). These shortcuts immediately show up as serious technical debt, leading to fragile software that is costly to maintain, susceptible to security attacks, and likely to suffer production outages.

4. Empirical Software Economics (Boehm’s 100 × Multiplier):

Empirical research in software engineering economics demonstrates that the cost to fix an error or architectural omission grows by a factor of 10× to 100× from the requirements/architecture phase to the operational deployment phase (Boehm, 1981; Boehm and Basili, 2001). Correcting an architectural diagram or decision record during Milestone 1 takes hours of analysis and involves no code rework cost; fixing the same architectural omission once code, tests, and deployment scripts are in place requires weeks of refactoring, live data migration, and emergency regression cycles.

As summarised in landmark architectural research: *"When an engineering decision is deferred until implementation, the decision is no longer made through proactive design; it is dictated by the accidental constraints of whatever code happened to be written first"* (Bass, Clements and Kazman, 2021, p. 28).

Section 3: Question 3 - Evidence-Based Technology Stack Decisions [15 Marks]

3.1 Overview of Candidate Contemporary Technology Stacks

3.1.1 Stack Approach A: Modern .NET ecosystem (.NET 8)

Frontend: TypeScript

Backend: .NET 8 (C#) Web API

Database: Microsoft SQL Server

Primary Hosting environment: Microsoft Azure

3.1.2 Stack Approach B: MERN Stack

Frontend: React.js (JavaScript)

Backend: Node.js runtime with Express.js Framework

Database: MongoDB (NoSQL)

Primary Hosting environment: AWS with database on MongoDB Atlas

3.2 Comparative Evaluation Against the Seven Required Lifecycle Criteria

3.2.1 Team capability and learning curve

.NET requires knowledge of C# and OOP principles to be able to use it efficiently. If you and/or your team already have knowledge in these areas the learning curve lessens drastically. .NET architecture in one of your programs can be applied in another .NET program. (Baid and Gupta, 2024)

MERN is easy to learn as you only write one language (JavaScript). There is many resources and tutorials available online to create an even easier learning experience making MERN very startup friendly.

3.2.2. Security and ecosystem

ASP.NET Core natively provides ASP.NET Core Identity, anti-forgery tokens for CSRF prevention, automatic output coding to prevent script injection and Data Protection API for encrypting and decrypting sensitive data. (Dinesh Gabhane, 2025) .NET core security middleware is built and maintained by Microsoft.

For an Azure hosted platform, it provides a single secure entry point for client traffic which can filter requests through the Azure firewall. It has end-to-end TLS encryption for data in transit and possesses logical network resource grouping and isolation through network segmentation. (Microsoft, 2025)

Standard features in .NET like CSRF tokens etc. must be imported manually in Express. Packages like helmet, express-validator must be added manually to prevent security vulnerabilities like XSS and clickjacking, provide authentication and prevent NoSQL

injection against MongoDB. (Das, 2024) MERN application relies on open-source npm modules which carries a significant risk. A vulnerability in deeply nested dependencies exposes the entire application. MongoDB is secured by IP whitelisting in Atlas or configuring virtual private clouds, so it only accepts connection from the Express API Server.

3.2.3 Maintainability

.NET consists of strongly typed C# with relational databases. Microsoft also provides long term support releases for up to 2 years making maintenance easier and safer.

MERN apps rely on micro-packages from third parties (npm). This can make the maintenance of code harder than a .NET stack as these packages need to be upgraded manually to ensure app security. MERN needs individual patches and upgrades so it will take more care to keep it to standard.

3.2.4. Testing and Tool Support

.NET provides good performances and integrated tools via Visual Studio and JetBrains Rider. Most tools needed for debugging and testing are already available through these IDE add-ins. .NET also has excellent integration with SQL Server and other Microsoft technologies. (Tanna, 2023)

MERN relies on a decentralized, modular ecosystem for testing and tooling. The testing process ensures that each component is working correctly individually as well as their interactions. The components are tested separately using tools like Jest for frontend or Cypress for end-to-end testing

3.2.5. Deployment and Platform Capability

Modern .NET stack is open source and is fully cross-platform. It is highly flexibility and can run on Windows, Linux or macOS, however older .NET frameworks were only able run on Windows.

MERN runs its Node.js and React containers across cloud platforms like AWS and Docker. It can use specialised hosting platforms or a single unified cloud service.

3.2.6. Development and likely Operational costs

.NET is free and open source, but the cost of development of applications can vary depending on the size of the project. A small to medium sized web-based application will commonly cost between R100 000 – R250 000 while a larger web-based application can cost over R1 million. This is dependent on the features of required and time it would take to develop. (Gleb Barkov, 2026)

MERN stacks often have similar, but slightly lower price ranges to the .NET. MERN stacks also have the advantage of only needing one developer for small scale projects as the front-end and back-end both use JavaScript which could save money.

3.2.7. Future change and maintainability implications

.NET stack is maintained by Microsoft with regular patches and updates. Upgrades are backwards compatible. The compiler will catch errors before the app goes live after refactoring. C# code rules ensure that large projects remain organised. .NET also forces OOP and dependency injection, so it is resistant to rotting over time. The apps follow a standard structure and are easy for other developers to pick up.

For MERN stacks, it relies on npm packages which can introduce security risks if not regularly checked and upgraded. Node and Express provide the Developers with a lot of flexibility, however if they don't maintain a strict structure to their code, it will be difficult to maintain it in future. .NET has much better future reliability but will be slower than MERN.

3.3 Justified Weighted Decision Matrix

Criteria	Weight	Justification	Stack A Score (10)	Stack A Weight	Stack B score (10)	Stack B Weight
Team capability and learning curve	0.15	Knowledge requirements and learnability impact the initial development speed and project delivery time	7	1.05	9	1.35
Security and ecosystem	0.16	Web-based systems face constant security threats which make protection and dependency critical	9	1.44	6	0.96
Maintainability	0.18	Maintainability is crucial. Refactoring code determines majority of the cost during the systems lifecycle	9	1.62	6	1.08
Testing and tooling support	0.12	Directly determines the release stability of the system	9	1.08	7	0.84
Deployment/platform compatibility	0.12	Cross-platform support and container efficiency effect the hosting flexibility of the system and the infrastructure setup costs	8	0.96	9	1.08

Development and likely operational cost	0.11	Cloud resource consumption and developer talents and skills directly dictate the overall cost	8	0.88	8	0.88
Future change and maintainability implications	0.16	Framework stability and long-term support cycles protect system against depreciation	9	1.44	6	0.96
TOTALS:	1.00	-	59	8.48	51	7.15

3.4 Criterion Weight Justifications & Evidence-Backed Scoring

3.4.1. Team capability and learning curve (Weight: 0.15)

Stack A (.NET 8) Score — 7.0/10: Requires developers to learn or have knowledge of C# for backend while using JavaScript/CSS/HTML or TypeScript for the frontend. C#'s object-oriented structure is more difficult to learn for developers without a prior knowledge of C#.

Stack B (MERN) Score — 9.0/10: Uses a single language (JavaScript) across both client (React) and server (Node.js/Express) layers. This allows a single developer with sufficient knowledge in JavaScript to be able to handle both frontend and backend

3.4.2. Security and ecosystem (Weight: 0.16)

Stack A (.NET 8) Score — 9.0/10: Microsoft provides security features built directly into ASP.NET Core. These features handle many security risks which minimizes third-party vulnerability exposure.

Stack B (MERN) Score — 6.0/10: MERN relies on open-source npm. The nested dependencies of npm packages expose applications to significant supply-chain risks and dependency injection. The packages must be updated manually or managed using third-party tools like npm audit.

3.4.3. Maintainability (Weight: 0.18)

Stack A (.NET 8) Score — 9.0/10: C# provides strict static typing with Object-Oriented coding practices. .NET enforces strict data schemas which makes refactoring on a large scale easier later. Microsoft also release frequent updates which are backwards compatible.

Stack B (MERN) Score — 6.0/10: MERN relies on JavaScript and MongoDB's schema-less architecture. It is highly flexible during initial development, however unmanaged JavaScript base structures can lead to run-time type errors, and "schema drift" in MongoDB which can cause data integrity errors if not checked

3.4.4. Testing and tooling support (Weight: 0.12)

Stack A (.NET 8) Score — 9.0/10: Offers good, integrated tooling through Visual Studio and JetBrains Rider. Includes testing and diagnostics

Stack B (MERN) Score — 7.0/10: Has flexible testing tools available inside of VS Code, however testing requires third-party libraries to be manually configured.

3.4.5. Deployment/platform compatibility (Weight: 0.12)

Stack A (.NET 8) Score — 8.0/10: Fully cross-platform and open-source runtime running on Linux, macOS, and Windows.

Stack B (MERN) Score — 9.0/10: Node.js runtime environment cross-platform primarily using cloud based platforms.. It can use specialised hosting platforms or a single unified cloud service.

3.4.6. Development and likely operational cost (Weight: 0.11)

Stack A (.NET 8) Score — 8.0/10: Modern .NET is completely open-source under the .NET Foundation. Excellent CPU/RAM throughput per node reduces cloud hosting resource consumption under heavy workloads, balancing potential specialist developer costs.

Stack B (MERN) Score — 8.0/10: MERN is open source. There are many JavaScript developers which keeps recruitment costs low, though unstructured MongoDB instances can have high hosting costs on a larger scale.

3.4.7. Future change and maintainability implications (Weight: 0.16)

Stack A (.NET 8) Score — 9.0/10: Maintenance supported by Microsoft regularly. Microsoft guarantees Long-Term Support (LTS) releases for up to 3 years per version and is backwards compatible.

Stack B (MERN) Score — 6.0/10: The Node.js and Express provide flexibility, but without structure can cause high maintenance costs in future. Dependency on third-party packages can cause depreciation and security risks if not managed or upgraded regularly.

3.5 Critical Analysis: Why the "Most Popular" Stack is Not Universally Optimal

A decision matrix is helpful but its is not absolute. The requirements of the specific project must be considered when making a choice of the appropriate stack

At first glance at the decision matrix the Stack A ranks higher than Stack B, but if criteria or weights change the “better” choice will also change. If the team working on a project has no prior C# experience and the deadline is in 3 weeks, the weighting of relevant criteria will change making Stack B the more appropriate choice. The nature of the enterprise requiring your system can also change the weight of your criteria. If the business is a medical institute dealing with sensitive client information security will be severely important.

Taking these factors into example a generic decision matrix cannot fit for all projects and the weight of criteria of your stack will change based on the crucial requirements of the project.

Section 4: Question 4 - Responsible AI in Software Engineering [10 Marks]

4.1 Three High-Value Engineering Use Cases for Generative AI

4.1.1 Automated Test Case Generation (Software Testing)

AI tools can be used to generate unit tests or mock datasets specific to your project to ensure that it is running smoothly and check edge cases. This will greatly reduce the workflow of the project by generating tests. The AI might also catch small errors or unexpected formats that a human developer might overlook. (Coursera, 2024) This will also improve long-term deployment by reducing potential future maintenance.

4.1.2. Performing Repetitive and Time-consuming Tasks

Updating README files and API documentation in larger systems can often get neglected or overlooked due to its time-consuming and repetitive nature. AI can generate standardized documentation and API specifications automatically allowing developers to focus on more crucial components.

4.1.3 Natural language to executable logic

AI can be used to take the requirement of the business and turn them into a base structure for your system. The developers can save time using AI to translate unstructured requirements into executable software structure to provide a faster development start. These can provide developers with base entities and relationships that can be expanded on later if necessary.

4.2 Three Critical Engineering Risks & Failure Modes

The use of AI during the engineering process could create many significant issues if not used correctly and responsibly. (Pearce H, et al, 2022)

4.2.1 Overreliance

As people use AI during the process of the project, they could completely rely on the output of the model for every process causing the engineer to not be able to sufficiently give evidence, track or even troubleshoot issues. This could cause incredible technical debt as changes are enacted wildly and without proper control or approval.

4.2.2 Situation consideration

The AI model might not have a complete idea of the wider project environment and will subsequently suggest or contribute things wildly incorrect and unsuitable for the stakeholders or create a vulnerability in the core structure already accounted for within the documentation.

4.2.3 Security

If you are inputting client data into AI models, it is not guaranteed to be secure within that model and could be extracted violating privacy laws. Additionally, there is a strong probability that the AI model will create code that is vulnerable to attacks and malicious actors.

4.3 Human-in-the-Loop Quality Control & Verification Protocol

Before work that is generated by AI is introduced into the project there should be at least 2 different team members there to approve and validate the rules that this piece of work is doing. Each of those two should make documentation that goes over what they found and what they would consider changing or what they think work for this piece of work.

4.4 The Accountability Defense: Engineering, Ethical, and Legal Responsibility

In the case of the following situation: “If an engineer accepts AI-generated output that later introduces a defect, vulnerability or incorrect engineering decision, who remains accountable and why?”

The primary person accountable would be the engineer but even before the defect became an issue the other team members should have seen the proposed change and saw that it had a defect. Largely the issue is with the organisational structure and maintaining correct and peer reviewed work throughout the project should be the primary quality consideration.

Section 6: Appendix - AI Research & Verification Record

Date	Team Member	AI Tool & Version	Purpose / Task	Output Used ?	How independently verified
26-09-01	Chris Fourie	Gemini 3.7 Flash	Initial Literature & Source Discovery: Identifying seminal academic papers, standards, and case study reports on software economics, modularity, and the Boeing 737 MAX MCAS failure.	Yes (Modified, swapped some suggestions out for others found)	Every suggested source was manually cross-checked via Google Scholar to verify author names, publication years, and actual content.
2026-09-02	Chris Fourie	Gemini 3.7 Flash	Decision Modelling & Causal Chain Synthesis: Scaffolding the structural format of the Decision Horizon Map and mapping the	Yes (had to manually create the diagram in online app)	Evaluated against Bass, Clements & Kazman (2021) architectural definitions and Boehm's Cost of Change curve.

References

1. **ACM / IEEE-CS Joint Task Force on Software Engineering Ethics.** (2020). *Software Engineering Code of Ethics and Professional Practice*. New York / Piscataway, NJ: ACM & IEEE. Available at: <https://www.acm.org/about-acm/code-of-ethics> [Accessed 1 September 2026].
2. **Bass, L., Clements, P. and Kazman, R.** (2021). *Software Architecture in Practice*. 4th edn. Boston: Addison-Wesley Professional.
3. **Beyer, B., Jones, C., Petoff, J. and Murphy, N.R.** (2016). *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA: O'Reilly Media.
4. **Boehm, B.W.** (1981). *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice-Hall.
5. **FAA (Federal Aviation Administration).** (2020). *Summary of the FAA's Review of the Boeing 737 MAX: Return to Service Decisions*. Washington, D.C.: U.S. Department of Transportation. Available at: https://www.faa.gov/foia/electronic_reading_room/boeing_reading_room/ [Accessed 1 September 2026].
6. **Fowler, M.** (2018). *Refactoring: Improving the Design of Existing Code*. 2nd edn. Boston: Addison-Wesley.
7. **Humble, J. and Farley, D.** (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Upper Saddle River, NJ: Addison-Wesley.
8. **IEEE Computer Society.** (2014). *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide V3.0)*. Piscataway, NJ: IEEE Computer Society Press.
9. **Kleppmann, M.** (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA: O'Reilly Media.
10. **Leveson, N.G.** (2020). 'Are We Learning from Software Failures? The Boeing 737 MAX Crashes', *MIT Aeronautics and Astronautics Technical Report*. Cambridge, MA: Massachusetts Institute of Technology.
11. **NIST (National Institute of Standards and Technology).** (2022). *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities*. NIST Special Publication 800-218. Gaithersburg, MD: U.S. Department of Commerce.
12. **OWASP Foundation.** (2021). *OWASP Top 10: 2021 — The Ten Most Critical Web Application Security Risks*. Available at: <https://owasp.org/Top10/> [Accessed 1 September 2026].
13. **Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B. and Karri, R.** (2022). 'Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions', in *2022 IEEE Symposium on Security and Privacy (SP)*. San Francisco, CA: IEEE, pp. 754–768.

14. **Wiggins, A.** (2017). *The Twelve-Factor App*. Available at: <https://12factor.net/> [Accessed 1 September 2026]
15. **Johnston, P. and Harris, R.,** (2019). The Boeing 737 MAX saga: lessons for software organizations. *Software Quality Professional*, 21(3), pp.4-12.
16. Baid, J. and Gupta, S. (2024). Comparative analysis of MERN, MEAN and MEVN stacks in web development. *INTERANTIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING AND MANAGEMENT*, 08(02), pp.1–10. doi:10.55041/ijsrem28539.
17. Coursera (2024). *20 Examples of generative AI applications across industries*. Coursera. Available at: <https://www.coursera.org/articles/generative-ai-applications>
18. Das, A. (2024). *What are the best practices for securing express.js applications in production?* Medium. Available at: <https://article.arunangshudas.com/what-are-the-best-practices-for-securing-express-js-applications-in-production-a8cff9955860>.
19. Dinesh Gabhane (2025). *Built-in security in ASP.NET core*. [online] C-sharpcorner.com. Available at: <https://www.c-sharpcorner.com/article/built-in-security-in-asp-net-core>.
20. GeeksforGeeks (2024). *Introduction to testing in MERN*. GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/mern/introduction-to-testing-in-mern/>.
21. Gleb Barkov (2026). *How to calculate the cost of software development*. Software development company | Diatom Enterprises. Available at: <https://diatomenterprises.com/blog/how-to-calculate-the-cost-of-software-development/>.
22. Microsoft (2025). *ASP.NET core security topics*. Microsoft.com. Available at: <https://learn.microsoft.com/en-us/aspnet/core/security/?view=aspnetcore-10.0>.
23. Tanna, D. (2023). *.NET core vs MERN Stack*. VIPL Insights. Available at: <https://blog.varianceinfotech.com/net-core-vs-mern-stack/> [Accessed 3 Sept. 2026].
24. Yadav, S. (2026). *Why I bet my career on .NET instead of the MERN stack in 2026*. Medium. Available at: <https://medium.com/@PragmaticPro/why-i-bet-my-career-on-net-instead-of-the-mern-stack-in-2026-f60e176c3c59>.
- 25.